University of Nairobi
Faculty of Science and Technology
Department of Physics

Design and Implementation of an Edge-AI Weather-Based Smart Irrigation Control System Using ESP32 Microcontrollers

by

Victor Stewart Ngunyi

I39/2920/2021

A Project Report Submitted in Fulfillment of the Requirements for the award of the Degree of

Bachelor of Science in Physics of the University of Nairobi

May, 2026

DECLARATION

I declare that this project is my original work and has not been submitted elsewhere for research. Where other people's work has been used, this has properly been acknowledged and referenced in accordance with the University of Nairobi's requirements.

Victor Stewart Ngunyi
I39/2920/2021
Department of Physics
University of Nairobi
victorstew5@students.uonbi.ac.ke

Signature ..... Date.....

This project report is submitted for examination with our approval as research supervisors:

Supervisor 1
University of Nairobi
Department of Physics
P.O. Box 30197 - 00100,
Nairobi, Kenya
supervisor-email@uonbi.ac.ke

Signature ..... Date.....

Supervisor 2
University of Nairobi
Department of Physics
P.O. Box 30197 - 00100,
Nairobi, Kenya
supervisor-email@uonbi.ac.ke

Signature ..... Date.....

ABSTRACT

Efficient irrigation requires knowledge of both present crop water demand and near-term rainfall conditions. Many low-cost automated irrigation systems rely on fixed thresholds or cloud-connected controllers, which may waste water when rainfall is imminent, fail where internet access is intermittent, or become difficult to scale across independent farm sections. This project designed and implemented a prototype edge-AI smart irrigation control system based on ESP32 microcontrollers, ESP-NOW peer-to-peer communication, and an embedded rainfall forecasting model. Historical hourly weather data for a representative central Kenya farm location at latitude -0.4201 and longitude 36.9476 were obtained from the Open-Meteo historical weather API for the period 1 January 2015 to 31 December 2025. The dataset contained 96,432 hourly records with temperature, relative humidity, derived vapour pressure deficit, soil temperature, soil moisture, shortwave radiation, FAO reference evapotranspiration and precipitation.

Two rainfall amount models were developed in Python using TensorFlow: a lightweight multilayer perceptron using current weather features and a 1-D convolutional neural network using 24-hour feature sequences. The multilayer perceptron was selected for firmware deployment because it generated a compact TensorFlow Lite model suitable for TensorFlow Lite Micro on the ESP32 master node. On the held-out chronological test set, the deployed model achieved a mean absolute error of 0.136 mm and a root mean square error of 0.484 mm for hourly rainfall amount prediction. A separate 1-D convolutional experiment achieved a lower root mean square error of 0.415 mm and a higher coefficient of determination, but required sequence input handling and was not integrated into the final firmware. The completed prototype uses sensor nodes to transmit weather observations, including vapour pressure deficit computed from measured temperature and relative humidity, a section node to drive irrigation, pesticide and fertilizer actuators, and a master node to perform local inference and send control commands. Irrigation is inhibited when predicted rainfall exceeds 1.0 mm and is enabled under high evaporative demand condi-

tions using reference evapotranspiration, solar radiation and humidity thresholds. The project demonstrates that a low-cost microcontroller network can combine meteorological sensing, local machine-learning inference and actuator control for weather-aware irrigation without continuous cloud dependence.

TABLE OF CONTENTS

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Research Background

Water is one of the most important limiting resources in crop production. Irrigation improves reliability of production where rainfall is seasonal, unevenly distributed or insufficient during sensitive crop growth stages, but inefficient scheduling can lead to water loss, energy waste, nutrient leaching and crop stress. Traditional irrigation practices often depend on fixed time schedules or visual judgement by the farmer. Such methods are simple, but they do not always respond to rapid changes in weather, soil conditions or the probability of rainfall.

Reference evapotranspiration, ET_0, is a standard measure of atmospheric demand for water from a reference crop surface. The FAO Penman-Monteith method is widely used because it combines the effects of solar radiation, air temperature, humidity and wind into a physically meaningful estimate of evaporative demand (Allen *et al.*, 1998). In irrigation management, ET_0 helps estimate how much water has been lost from the crop-soil system, while rainfall information helps determine whether irrigation should be delayed or reduced. A practical irrigation controller therefore benefits from combining measured weather, evapotranspiration and short-term rainfall prediction.

Recent advances in Internet of Things systems have made it possible to collect farm data using inexpensive sensors and microcontrollers. Reviews and platform studies of smart irrigation systems show that low-cost sensors, embedded control, wireless communication and automated decision logic can improve the timeliness of irrigation and reduce water waste when compared with conventional scheduling (Garcia *et al.*, 2020; Kamienski *et al.*, 2019). However, many sys-

tems depend on continuous internet connectivity, remote cloud analytics or manually configured thresholds. These requirements can be a weakness in rural or semi-rural farms where network coverage, power and maintenance capacity may be limited.

This project addresses that challenge by implementing intelligence at the edge. The system uses ESP32 microcontrollers communicating through ESP-NOW, a connectionless peer-to-peer wireless protocol for Espressif devices (Espressif Systems, 2026). A master ESP32 runs an embedded TensorFlow Lite Micro rainfall model, processes weather data received from field nodes and sends actuator commands to a section controller. TensorFlow Lite Micro is designed for running neural network inference on microcontrollers with restricted memory and processing resources (David *et al.*, 2021). This makes it appropriate for a smart irrigation prototype where decisions should continue even without a cloud server.

1.2 Statement of the Problem

Small and medium-scale irrigation systems often lack affordable local intelligence. Manual irrigation and timer-based automation do not account for immediate weather conditions or expected rainfall. As a result, irrigation may occur shortly before rainfall, wasting water and energy. Conversely, crops may remain unirrigated during periods of high evaporative demand when human supervision is delayed. Commercial smart controllers may solve some of these problems, but they can be costly, cloud-dependent or poorly adapted to multi-section farm layouts.

The specific problem addressed in this project is the need for a low-cost, locally controlled irrigation system that can receive weather information from distributed sensor nodes, estimate near-term rainfall using an embedded machine-learning model and issue section-level actuator commands without requiring continuous internet connectivity.

1.3 General Objective

The overall objective of this project was to design and implement an ESP32-based edge-AI smart irrigation control system that uses local weather information and embedded rainfall forecasting to automate irrigation decisions.

1.4 Specific Objectives

(i) To obtain and preprocess historical hourly weather data suitable for rainfall forecasting and irrigation decision support.

(ii) To develop and evaluate lightweight machine-learning models for hourly rainfall amount prediction.

(iii) To convert the selected model to a TensorFlow Lite Micro-compatible format for deployment on an ESP32 microcontroller.

(iv) To design firmware for sensor, section and master ESP32 nodes communicating through ESP-NOW.

(v) To implement a control algorithm that combines predicted rainfall, ET_0, shortwave radiation and relative humidity to decide when irrigation should be activated.

(vi) To evaluate the performance, limitations and future improvement areas of the developed prototype.

1.5 Justification and Significance of the Study

Agricultural water management requires systems that are not only technically accurate, but also affordable and robust under field conditions. A locally controlled ESP32 system is significant because ESP32 boards are inexpensive, widely available and capable of both wireless communication and embedded computation. By using ESP-NOW, the controller can exchange data among farm nodes without relying on a router or cellular data link. By using TensorFlow Lite Micro, the rainfall model can run directly on the master node.

The project contributes to the growing field of precision irrigation by demonstrating a practical integration of weather-based crop water demand, machine-learning rainfall prediction and microcontroller actuation. The work is also relevant to physics because it uses physical environmental variables such as radiation, humidity, temperature and evapotranspiration, and applies modelling, signal processing and embedded instrumentation principles to a real agricultural problem.

1.6 Scope of the Study

The project focused on a prototype system for section-level irrigation control. The software and firmware were developed for three ESP32 roles: sensor node, section node and master node. Historical weather data from Open-Meteo were used for model training and testing. The deployed rainfall model predicts hourly rainfall amount from current weather variables; it does not yet use live soil moisture sensors, crop coefficients for a specific crop, wind speed, measured flow rate or feedback from an actual field water balance experiment. The actuator outputs include irrigation, pesticide and fertilizer channels, but the main analysis of this report focuses on irrigation control.

Chapter 2

Literature Review

2.1 Smart Irrigation and Precision Agriculture

Smart irrigation systems use sensors, controllers, communication networks and decision algorithms to apply water when and where it is needed. In recent literature, common measurements include soil moisture, temperature, humidity, radiation, water level and weather forecasts (Garcia *et al.*, 2020). These systems may be classified as soil-moisture based, weather-based, plant-stress based or hybrid. Soil-moisture systems respond directly to root-zone water availability, while weather-based systems estimate demand using evapotranspiration and rainfall.

The main advantage of smart irrigation is improved water use efficiency. A controller can avoid unnecessary irrigation after rainfall, irrigate during suitable times of day and apply water according to section-specific conditions. The SWAMP precision irrigation platform, for example, shows how sensing, communications, data management and decision algorithms can be integrated for farm water management while still facing deployment and scalability constraints (Kamienski *et al.*, 2019). However, the same literature also highlights recurring limitations: power management, communication reliability, system cost, sensor calibration and lack of locally adapted decision models.

2.2 Weather-Based Irrigation Scheduling

Weather-based irrigation scheduling relies on the relationship between atmospheric conditions and crop water loss. Solar radiation provides the energy for evaporation, temperature affects sat-

uration vapour pressure, humidity controls vapour pressure deficit and wind influences transport of water vapour away from the crop surface. The FAO Penman-Monteith framework combines these variables into ET_0, which can be multiplied by a crop coefficient to estimate crop evapotranspiration (Allen *et al.*, 1998). In a simplified prototype, ET_0 can be used as an indicator of evaporative pressure even when crop-specific coefficients are not available.

Rainfall is equally important. If rainfall is expected, irrigation can be postponed to avoid overwatering. Historical gridded and reanalysis weather products have become useful for model development where local station records are unavailable. ERA5 reanalysis provides long-term, physically consistent weather fields (Hersbach *et al.*, 2020), and Open-Meteo exposes historical weather variables through a public API suitable for application development (Open-Meteo, 2026). In this project, Open-Meteo was used to assemble local hourly data for training and testing rainfall models.

2.3 Machine Learning for Rainfall Forecasting

Rainfall forecasting is difficult because rainfall is intermittent, nonlinear and often highly localized. Traditional statistical models can capture seasonal or autoregressive patterns, while machine-learning models can learn nonlinear relationships among meteorological variables. Neural networks are commonly used because they can approximate complex input-output mappings when adequate data are available (Goodfellow *et al.*, 2016). In agriculture, machine learning has been applied to crop prediction, irrigation scheduling, soil moisture estimation and climate-based decision support (Liakos *et al.*, 2018).

For an embedded irrigation controller, model accuracy must be balanced against memory, computation and simplicity. A dense neural network using current weather variables is easier to deploy than a sequence model because it requires only one feature vector at inference time. A convolutional neural network can capture temporal patterns over previous hours (LeCun *et al.*, 1998), but it requires buffering and preprocessing a fixed-length sequence. This project therefore evaluated both a lightweight MLP and a 24-hour 1-D CNN, then selected the MLP for final firmware integration.

2.4 Edge AI and Microcontroller Deployment

Edge AI refers to running machine-learning inference on local devices rather than sending all data to a remote server. This reduces latency, internet dependency and data transfer requirements. TensorFlow Lite Micro supports neural network inference on microcontrollers by using a small runtime, a static tensor arena and operators selected at compile time (David *et al.*, 2021). These design features are important for ESP32-based prototypes where memory is measured in kilobytes rather than gigabytes.

Deploying a model on a microcontroller involves more than training accuracy. The model must be converted to a compact inference format, embedded in firmware, supplied with the same feature scaling used during training and tested against memory limits. In this project, the selected rainfall model was exported as a TensorFlow Lite file and converted into C/C++ arrays included by the master ESP32 firmware.

2.5 Wireless Communication for Farm Nodes

Farm nodes require low-cost communication between sensors and controllers. Wi-Fi, LoRa, Zigbee, Bluetooth Low Energy and ESP-NOW are common options. ESP-NOW is useful for small ESP32 networks because it supports direct peer-to-peer communication using MAC addresses and does not require a wireless router (Espressif Systems, 2026). It is therefore suitable for a prototype where sensor nodes, section nodes and a master controller exchange short structured packets.

The limitation of ESP-NOW is that network management, peer registration, packet acknowledgement and security must be carefully handled in firmware. Unlike a full IP network, application-level packet formats and routing logic must be designed by the developer. This project implemented a shared packet structure with message types for weather data, control commands, status and acknowledgements.

2.6 Identified Knowledge Gap

The literature shows many examples of smart irrigation, but there remains a practical gap between cloud-based intelligent systems and simple local timer systems. Farmers need systems that are affordable, section-aware, weather-aware and able to operate locally. This project responds

to that gap by combining ESP-NOW communication, embedded rainfall inference and ET_0-based decision logic in a working prototype architecture.

Chapter 3

Theory

3.1 Crop Water Balance

Irrigation scheduling can be described using a simplified root-zone water balance. If D_t is the soil water deficit at time t, the deficit can be approximated by

$$D_t = D_{t-1} + ET_c - P_e - I, \tag{3.1}$$

where ET_c is crop evapotranspiration, P_e is effective rainfall and I is irrigation. When evapotranspiration is high and rainfall is low, the deficit increases and irrigation may be required. When rainfall is expected, irrigation may be delayed because rainfall contributes to reducing the deficit.

Crop evapotranspiration is commonly estimated as

$$ET_c = K_c ET_0, \tag{3.2}$$

where K_c is a crop coefficient and ET_0 is reference evapotranspiration. This project did not implement crop-specific K_c values; instead, ET_0 was used directly as a practical indicator of atmospheric water demand.

3.2 FAO Penman-Monteith ET_0

The FAO-56 Penman-Monteith equation estimates daily reference evapotranspiration as (Allen *et al.*, 1998)

$$ET_0 = \frac{0.408\Delta(R_n - G) + \gamma \frac{900}{T+273} u_2 (e_s - e_a)}{\Delta + \gamma(1 + 0.34u_2)}, \tag{3.3}$$

where R_n is net radiation, G is soil heat flux density, T is mean air temperature, u_2 is wind speed at 2 m height, $e_s - e_a$ is vapour pressure deficit, Δ is the slope of the saturation vapour pressure curve and γ is the psychrometric constant. The Open-Meteo dataset used in this project supplied hourly FAO ET_0, allowing the controller to use evapotranspiration without calculating all intermediate meteorological terms in the firmware.

Vapour pressure deficit was also used directly as a model input. It was calculated from air temperature and relative humidity as

$$e_s = 0.6108 \exp\left(\frac{17.27T}{T + 237.3}\right), \tag{3.4}$$

$$VPD = e_s \left(1 - \frac{RH}{100}\right), \tag{3.5}$$

where e_s is saturation vapour pressure in kPa, T is air temperature in degrees Celsius and RH is relative humidity in percent. This derived feature summarizes atmospheric drying demand using measurements already available at the sensor node.

3.3 Rainfall Prediction Model

The deployed rainfall model predicts hourly rainfall amount from a feature vector

$$x = [T,\ RH,\ VPD,\ T_s,\ M_s,\ R_s,\ ET_0], \tag{3.6}$$

where T is air temperature, RH is relative humidity, VPD is vapour pressure deficit, T_s is soil temperature, M_s is near-surface soil moisture, R_s is shortwave radiation and ET_0 is reference evapotranspiration. The features are standardized using training-set statistics:

$$x_i' = \frac{x_i - \mu_i}{\sigma_i}. \tag{3.7}$$

Because rainfall amount is non-negative and skewed, the training target was transformed using

$$y' = \frac{\ln(1+y) - \mu_y}{\sigma_y}. \tag{3.8}$$

After inference, the firmware reverses the transform using

$$\hat{y} = \exp(\hat{y}'\sigma_y + \mu_y) - 1. \tag{3.9}$$

The MLP used in this project can be represented as

$$h_1 = f(W_1 x' + b_1), \tag{3.10}$$

$$h_2 = f(W_2 h_1 + b_2), \tag{3.11}$$

$$\hat{y}' = W_3 h_2 + b_3, \tag{3.12}$$

where f is the rectified linear unit activation function. The model was trained by minimizing mean squared error on the transformed rainfall target.

3.4 Model Evaluation Metrics

Three metrics were used to evaluate rainfall amount prediction. Mean absolute error is

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|. \tag{3.13}$$

Root mean square error is

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}. \tag{3.14}$$

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}. \tag{3.15}$$

MAE is easier to interpret in millimetres, RMSE penalizes larger errors more strongly and R^2 indicates how much variance is explained relative to a mean baseline.

3.5 Control Logic

The irrigation decision in the firmware is a hybrid of machine-learning prediction and rule-based physics intuition. Irrigation is blocked if predicted rainfall exceeds a rainfall threshold. If rain is not expected, irrigation is enabled when evaporative demand and radiation are high and humidity is sufficiently low. The implemented decision can be summarized as:

$$\text{irrigate} = (\hat{P} < 1.0) \wedge (ET_0 \geq 0.20) \wedge (R_s \geq 300) \wedge (RH \leq 70). \tag{3.16}$$

When irrigation is enabled, the duration is scaled from 60 s to 300 s according to ET_0 between 0.20 mm and 0.60 mm. This approach is simple enough for a prototype while still linking control to predicted rainfall and atmospheric water demand.

Chapter 4

Methodology

4.1 Study Area and Data Source

The model development used a representative farm location in central Kenya with coordinates latitude -0.4201 and longitude 36.9476. Historical hourly weather records were downloaded from the Open-Meteo historical weather API, which provides historical weather variables derived from reanalysis and weather model datasets (Open-Meteo, 2026). The period covered was 1 January 2015 to 31 December 2025 in the Africa/Nairobi timezone.

The downloaded variables were air temperature at 2 m, relative humidity at 2 m, soil temperature from 0 to 7 cm, soil moisture from 0 to 7 cm, shortwave radiation, FAO ET_0 evapotranspiration and precipitation. Vapour pressure deficit was then derived from temperature and relative humidity. These variables were selected because they are physically connected to evaporation, soil water status, rainfall formation and irrigation need.

4.2 Research Design

The project followed a design-build-test approach. First, historical weather data were collected and normalized. Second, rainfall forecasting models were trained and evaluated. Third, the selected model was converted into a TensorFlow Lite format and exported into firmware source files. Fourth, ESP32 firmware was implemented for the sensor, section and master nodes. Finally, the complete system logic was evaluated using model metrics, firmware structure and prototype behaviour.

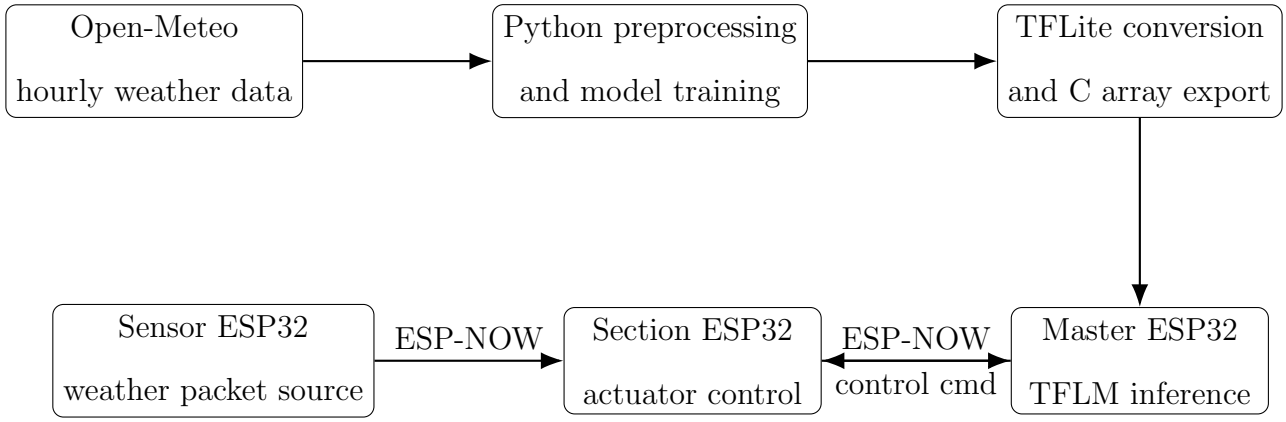

Figure 4.1: Overall workflow from weather data acquisition to embedded irrigation control.

4.3 Hardware and Software Materials

The main hardware platform was the ESP32 microcontroller. Three firmware roles were implemented:

(i) A sensor node that sends weather packets and heartbeat/status packets.

(ii) A section node that receives weather data from sensor nodes, forwards them to the master and drives actuator pins.

(iii) A master node that receives weather information, runs rainfall inference and sends irrigation commands.

The section node firmware defines GPIO outputs for a valve, pesticide sprayer and fertilizer actuator. In the prototype code these are assigned to pins 25, 26 and 27 respectively. The software tools used were Python, pandas, NumPy, scikit-learn, TensorFlow, TensorFlow Lite, Arduino-style ESP32 firmware and TensorFlow Lite Micro.

Image placeholder

Add image file: `images/esp32_prototype_overview.jpg`

Recommended evidence: a clear photograph of the assembled prototype showing the ESP32 boards, jumper connections, breadboard or carrier board, power supply and actuator interface.

Figure 4.2: Assembled ESP32 prototype used for the smart irrigation control system. The photograph should show the master, section and sensor-node hardware or a representative subset of the nodes used during testing.

Image placeholder

Add image file: `images/actuator_valve_setup.jpg`

Recommended evidence: a close-up photograph of the section-node actuator side, including relay module or driver circuit, valve connection and the GPIO wiring for irrigation, pesticide and fertilizer outputs.

Figure 4.3: Section-node actuator interface for farm control. The valve, spray and fertilizer outputs are controlled by GPIO pins on the ESP32 section node, allowing the master node to translate weather-aware decisions into physical action.

4.4 Field Sensor Inputs for Model Features

The deployed rainfall model expects seven weather inputs: air temperature, relative humidity, vapour pressure deficit, near-surface soil temperature, near-surface soil moisture, shortwave radiation and ET_0. In the prototype dataset these values came from Open-Meteo or were derived from Open-Meteo variables, but the same input vector can be produced from a field sensor node after calibration. The sensor node samples local instruments, converts raw readings into physical units, calculates vapour pressure deficit from temperature and humidity, and transmits the

resulting feature vector to the section node and master node through ESP-NOW.

An AHT10 temperature and humidity sensor can provide the air temperature and relative humidity terms used by the model. The sensor should be placed in a shaded, ventilated enclosure so that direct solar heating does not bias the temperature reading. These two measurements are important because temperature and humidity influence evaporation, vapour pressure deficit and rainfall likelihood. The firmware computes vapour pressure deficit in kPa from the same measurements, so the AHT10 output can replace the corresponding Open-Meteo columns during field operation, provided the firmware uses the same units as the training dataset.

Soil sensors provide two of the deployed model inputs. A soil temperature probe placed near the surface can provide the `soil_temperature_0_to_7cm` feature, while a calibrated capacitive or resistive soil moisture sensor can provide the `soil_moisture_0_to_7cm` feature. Soil moisture is useful because it gives the model and controller a direct indication of near-surface water status rather than relying only on atmospheric variables. After calibration against dry and saturated soil conditions, the same soil moisture reading can also be used as a control override when the soil is already wet.

Solar panels can support the system in two roles. First, they can power remote ESP32 sensor nodes through a charge controller and battery, allowing the field network to operate where mains power is unavailable. Second, the panel voltage, current or power output can act as a calibrated proxy for incoming solar radiation. This proxy should be treated carefully because a photovoltaic panel is not a pyranometer; panel temperature, angle, shading and battery charge state can affect the reading. For the prototype model, calibrated panel output or a dedicated light/irradiance sensor can be used to estimate the `shortwave_radiation` feature, which then contributes to ET_0 estimation and irrigation-demand decisions.

Table 4.1: Possible field sensors for generating model inputs.

Model input	Possible field source	Notes for use
Air temperature	AHT10	Mount in a shaded ventilated enclosure
Relative humidity	AHT10	Use percent relative humidity as in training data
Vapour pressure deficit	Sensor-node firmware	Compute from air temperature and relative humidity in kPa
Soil temperature	Soil temperature probe	Place near 0–7 cm depth to match dataset feature
Soil moisture	Capacitive or resistive soil sensor	Calibrate to match `soil_moisture_0_to_7cm`
Shortwave radiation	Calibrated solar panel or light sensor	Convert proxy reading to W m^{-2} if possible
ET_0	Firmware estimate or external weather source	Compute from temperature, humidity and radiation, or receive from API

4.5 Weather Data Processing

The data acquisition script downloaded hourly historical data and saved it as `historical_weather_data_hourly.csv`. The preprocessing steps were:

(i) Convert timestamps to local time.

(ii) Rename precipitation and evapotranspiration fields to consistent names.

(iii) Derive vapour pressure deficit from air temperature and relative humidity.

(iv) Clip negative precipitation values to zero.

(v) Drop records with missing input or target variables.

(vi) Sort records chronologically.

The final dataset contained 96,432 hourly records. The chronological split used 80% of the data for training and 20% for testing. Chronological splitting was selected to avoid training on future records when evaluating past predictions.

Table 4.2: Summary statistics of the historical hourly dataset.

Variable	Mean	Minimum	Maximum	Standard deviation
Air temperature (deg C)	16.853	7.100	29.400	3.495
Relative humidity (%)	77.778	16.000	100.000	16.320
Vapour pressure deficit (kPa)	0.499	0.000	3.326	0.478
Soil temperature 0–7 cm (deg C)	18.356	9.200	31.700	3.280
Soil moisture 0–7 cm (m^3 m^{-3})	0.316	0.112	0.479	0.082
Shortwave radiation (W m^{-2})	236.356	0.000	1143.000	315.409
ET_0 (mm h^{-1})	0.156	0.000	0.830	0.207
Precipitation (mm h^{-1})	0.167	0.000	17.200	0.600

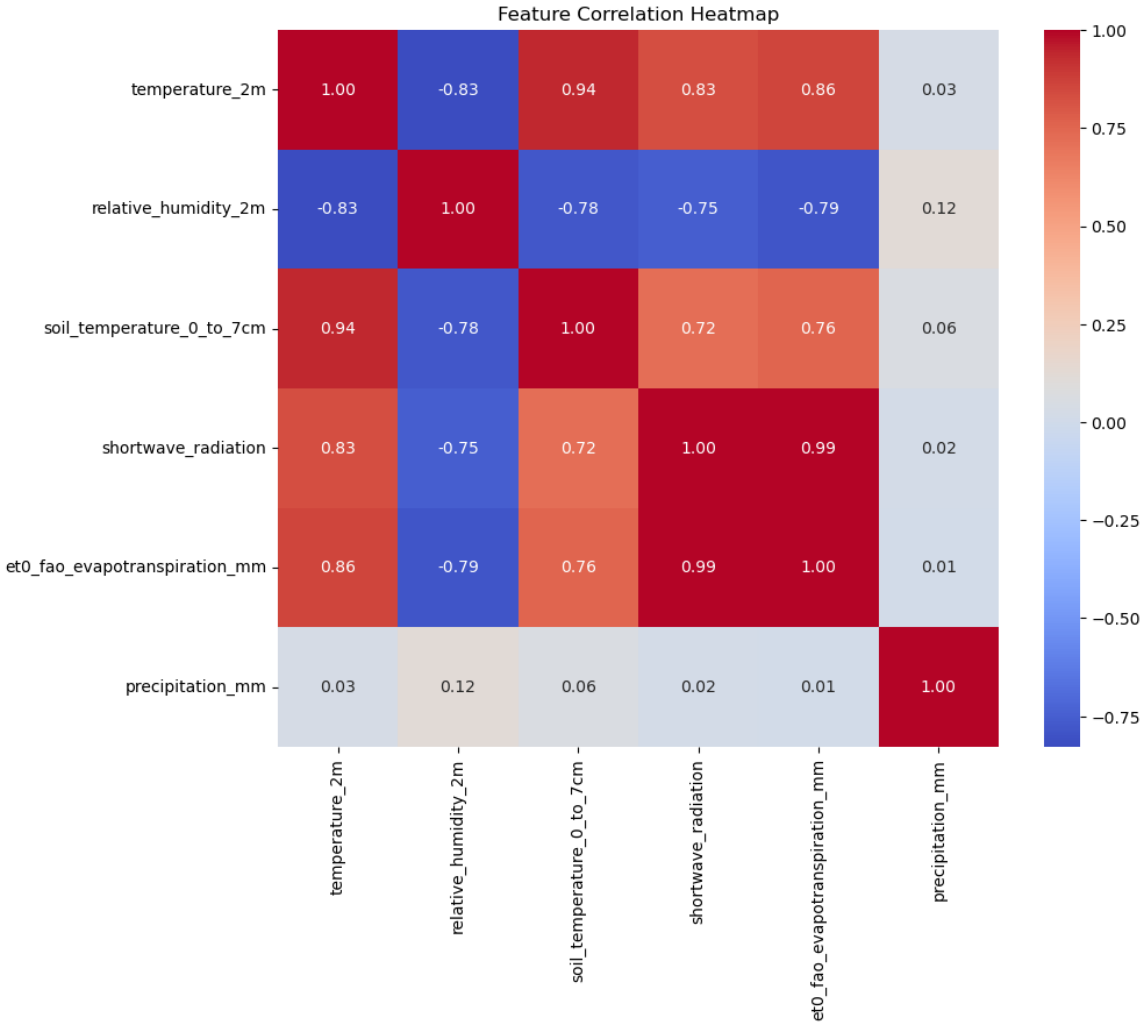

Figure 4.4: Feature correlation heatmap for the selected weather variables and precipitation. The heatmap is important for feature engineering because it shows which variables are physically related, which variables may carry overlapping information and how weak direct linear correlation with rainfall motivates the use of a nonlinear neural network instead of a simple linear rule.

The correlation heatmap should be used to discuss both relationships and limitations. Temperature, radiation and ET_0 are expected to move together because radiation and temperature increase evaporative demand. Humidity often moves in the opposite direction during hot, dry periods. Rainfall may show only weak linear correlation with any single input, which supports using the combined feature vector rather than relying on one threshold.

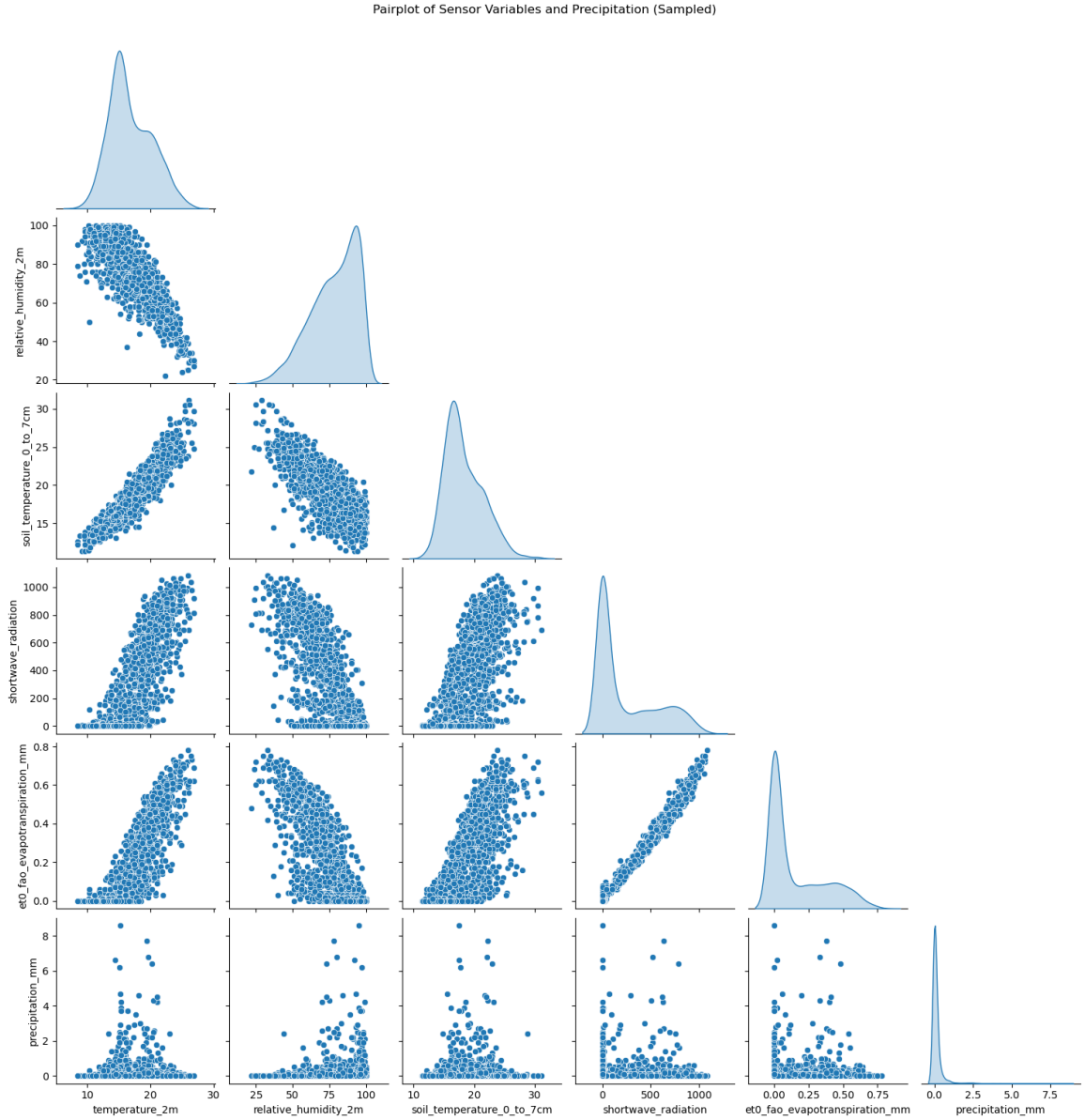

Figure 4.5: Pairplot of selected sensor variables and precipitation. This graph supports feature engineering by showing distributions, outliers, nonlinear relationships and the strong skew in precipitation values. It also helps confirm whether scaling, clipping and target transformation are needed before training.

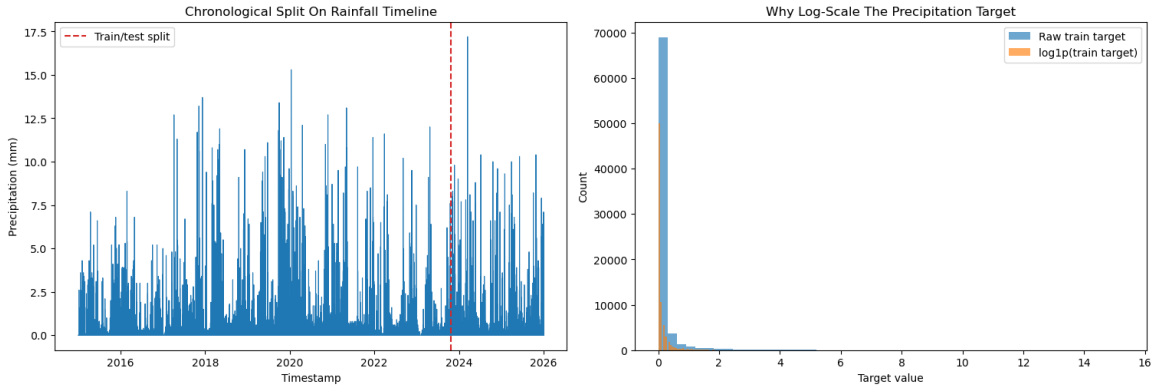

Figure 4.6: Chronological train-test split and rainfall target transformation. The timeline verifies that the model was evaluated on later unseen records, while the raw and log-scaled target histograms show why `log1p` was applied to reduce skew and make optimization less dominated by rare heavy-rainfall hours.

4.6 Rainfall Model Development

Two TensorFlow models were trained. The first was an MLP with two hidden layers of 32 and 16 neurons. The second was a 1-D CNN using 24-hour sequences. Both models used the same seven weather features and the same log-transformed rainfall target. Early stopping monitored validation loss with patience of five epochs, and training was limited to 40 epochs with a batch size of 256.

The MLP was selected for deployment because it only requires the latest feature vector. The CNN was retained as an experimental comparison because it tests the value of short history. The model training scripts also exported the feature mean, feature standard deviation, rainfall target mean and rainfall target standard deviation. These constants were written into the master firmware preprocessing header to ensure that the ESP32 used the same scaling as Python training.

4.7 Embedded Model Export

After training, the selected Keras model was converted to TensorFlow Lite. The TensorFlow Lite file was then converted into a C/C++ byte array included in the ESP32 master firmware. The generated files were:

- `hourly_rainfall_forecaster.cpp`
- `hourly_rainfall_forecaster.h`

- `hourly_rainfall_preprocessing.h`

The master firmware creates a TensorFlow Lite Micro interpreter, registers the fully connected operator, allocates a 48 KB tensor arena and checks the input shape before enabling inference. The input tensor receives standardized weather features, and the output tensor is converted back to rainfall in millimetres using the inverse log transform.

4.8 ESP-NOW Communication Design

All firmware roles use a shared packet structure containing source MAC address, destination MAC address, section ID, node ID, sequence number, message type and a union payload. The message types are weather data, control command, status and acknowledgement. This structure keeps packets small and deterministic.

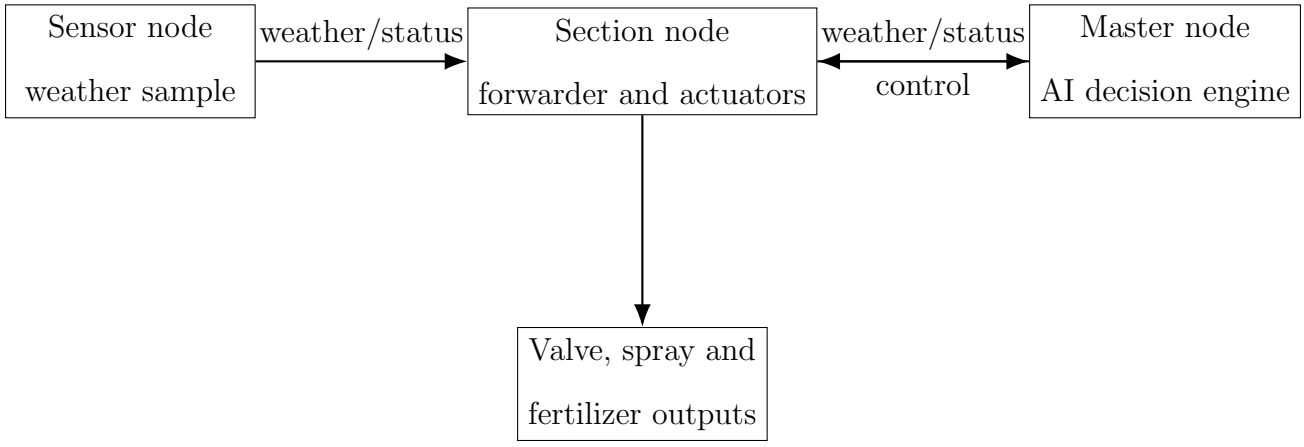

Figure 4.7: Three-role ESP32 network architecture implemented in the prototype.

Figure 4.8 shows how the same prototype architecture can be deployed on a farm. The master node may be placed in a protected central location, while section nodes are installed near valves or actuator boxes. Sensor nodes are distributed across the farm sections so that each section can report local weather or soil conditions. This arrangement keeps the control decision local and allows the system to expand by adding more sensor or section nodes.

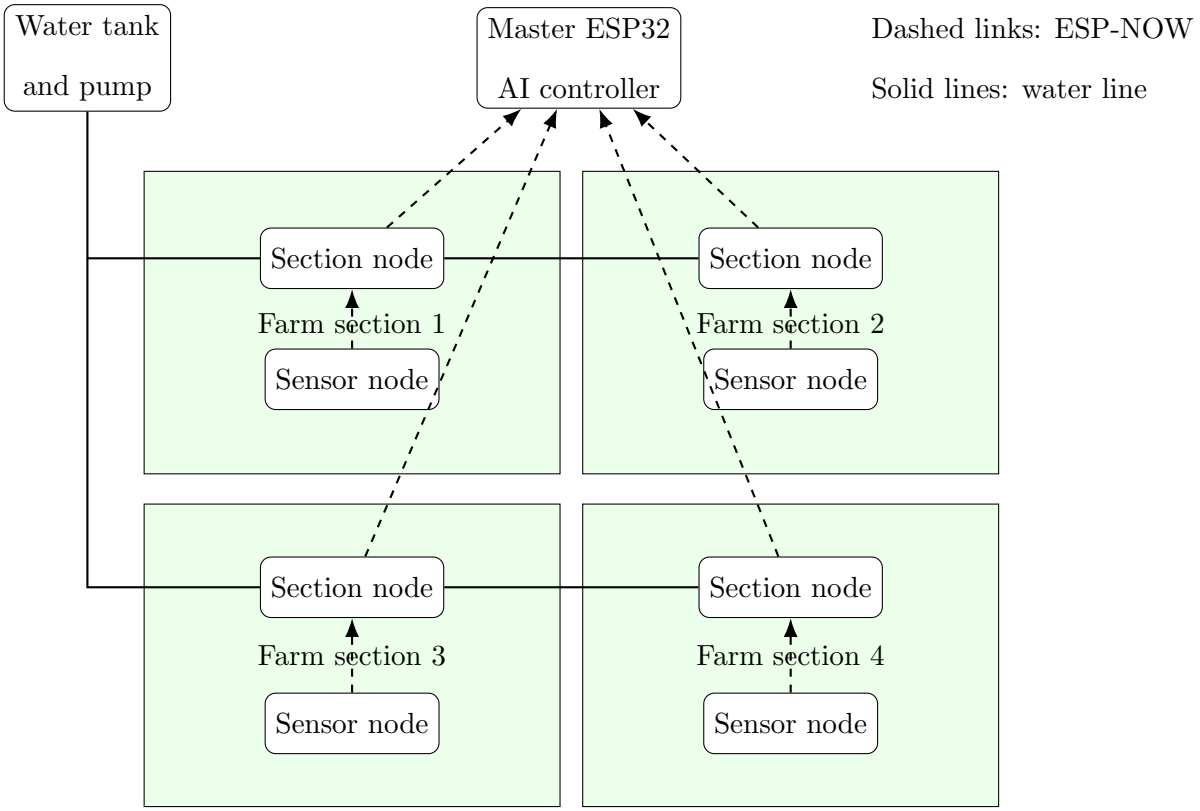

Figure 4.8: Recommended farm deployment layout for the smart irrigation system. Sensor nodes collect local field conditions, section nodes control valves and actuators, and the master node performs rainfall inference before sending section-specific control commands.

4.9 Control Algorithm

The master node runs a control cycle every 30 s. During each cycle it checks stored weather data for active sensor nodes, runs the rainfall model and sends a control packet to the relevant section node. The implemented thresholds are shown in Table 4.3.

Table 4.3: Irrigation decision thresholds implemented in the master firmware.

Parameter	Implemented value
Rainfall block threshold	Predicted rainfall ≥ 1.00 mm
Minimum ET_0 for irrigation	0.20 mm h^{-1}
Minimum shortwave radiation	300 W m^{-2}
Maximum relative humidity	70%
Minimum irrigation duration	60 s
Maximum irrigation duration	300 s
Control interval	30 s

Chapter 5

Results, Discussion, Conclusion and Recommendations

5.1 Dataset Results

The historical weather dataset covered eleven complete years from 2015 to 2025 and contained 96,432 hourly records. Rainfall was sparse and skewed: 34.27% of the hours had precipitation of at least 0.1 mm, while only 3.88% of hours had precipitation of at least 1.0 mm. This imbalance is typical of rainfall prediction and explains why simple accuracy is not a useful metric for rainfall amount modelling.

The dataset also showed strong seasonal rainfall variation. The wettest accumulated months in the 2015–2025 dataset were April, May, October and November, while the lowest accumulated rainfall occurred around July, August and September. This seasonal structure supports the use of meteorological features and historical data for rainfall-aware irrigation decisions.

Table 5.1: Annual precipitation totals in the processed dataset.

Year	Total rainfall (mm)	Year	Total rainfall (mm)
2015	1732.4	2021	1463.4
2016	1276.0	2022	912.1
2017	1168.0	2023	1049.6
2018	1773.7	2024	1134.8
2019	2458.8	2025	1165.8
2020	1929.0		

5.2 Rainfall Model Performance

Table 5.2 shows the test results for the two trained rainfall amount models. The CNN achieved the lower MAE, lower RMSE and higher R^2, suggesting that previous weather conditions contain useful information for rainfall amount prediction. However, the MLP was selected for firmware deployment because it was smaller and required only a single weather sample at inference time.

Table 5.2: Rainfall amount model performance on the chronological test set.

Model	MAE (mm)	RMSE (mm)	R^2	Rainy MAE (mm)	Rainy RMSE (mm)
MLP current-weather model	0.136	0.484	0.284	0.327	0.869
1-D CNN 24-hour sequence model	0.137	0.415	0.475	0.309	0.730

The R^2 values are modest, which is expected for hourly rainfall amount prediction using only seven local weather variables and no radar, satellite imagery, pressure fields or numerical weather forecast sequences. The model is therefore better interpreted as a practical rainfall-risk signal for irrigation control rather than a complete meteorological forecasting system. In the control context, even a coarse rainfall amount estimate is useful because the controller mainly needs to decide whether irrigation should be blocked when rain is likely.

The notebook evaluation graphs should be included to complement the numerical metrics in Table 5.2. MAE and RMSE summarize average error, but the plots show whether the model follows the shape of rainfall events, whether errors are biased and whether the model is useful for the control decision of distinguishing rain from no-rain conditions.

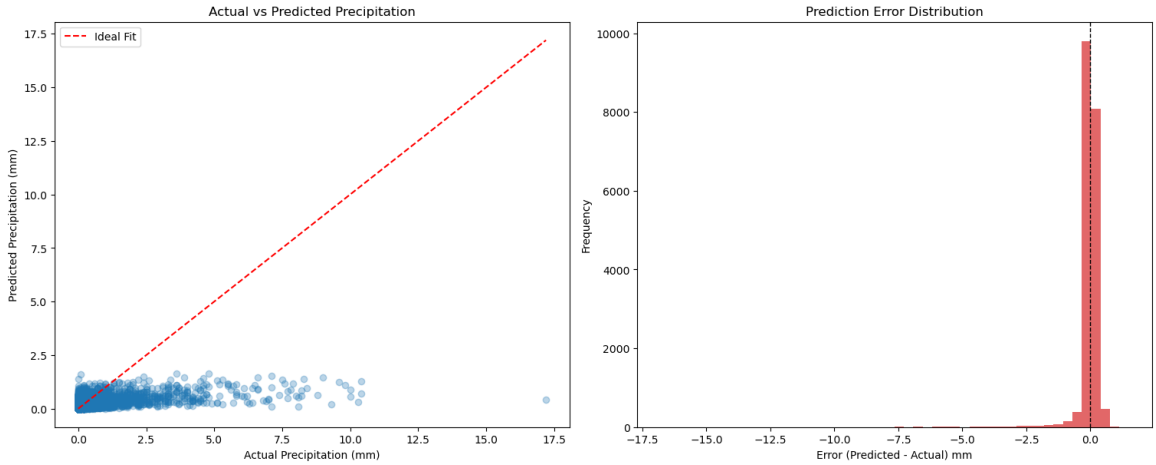

Figure 5.1: MLP actual-versus-predicted rainfall and prediction error distribution. The scatter plot shows how close the model is to the ideal one-to-one line, while the error histogram reveals bias, spread and the frequency of large under-predictions or over-predictions. This is important because irrigation control is sensitive to missed rainfall events and large false rainfall estimates.

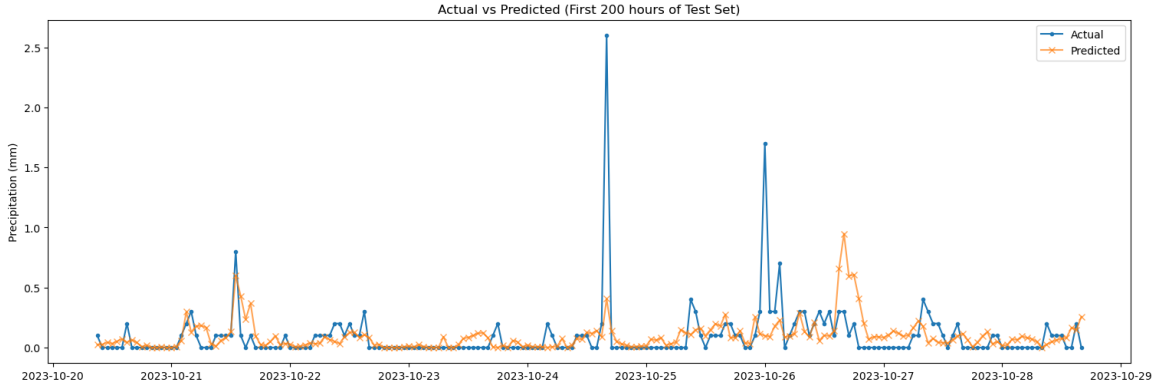

Figure 5.2: MLP actual and predicted rainfall over the first 200 hours of the test set. A time-series comparison is useful because rainfall forecasting is sequential in real use; it shows whether the model reacts during wet periods, whether it predicts rain during dry periods and whether peak rainfall amounts are underestimated.

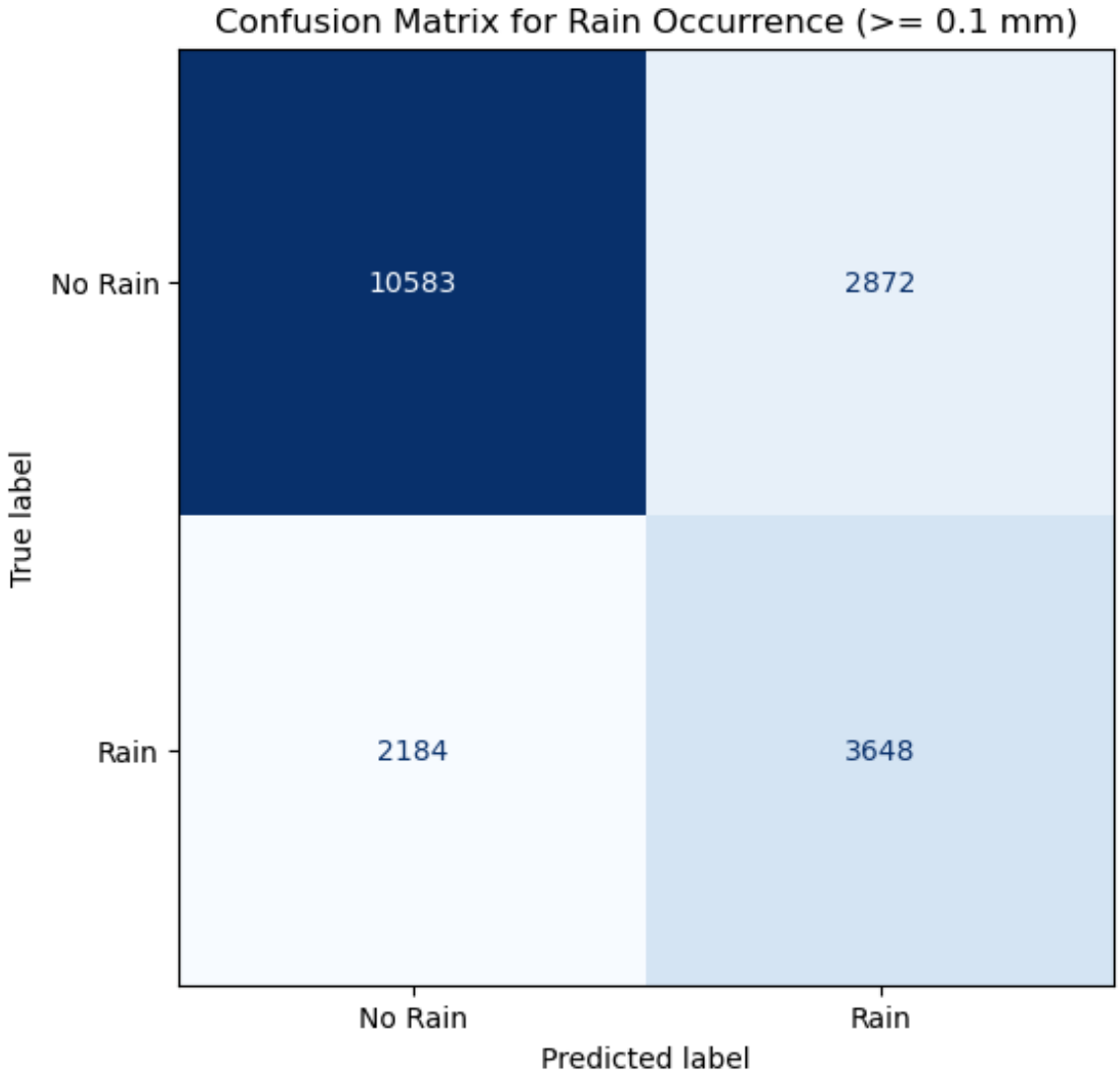

Figure 5.3: MLP rain-occurrence confusion matrix. This graph translates rainfall amount prediction into a control-oriented question: did the model identify rain or no rain? It is significant for irrigation because false negatives can lead to watering before rainfall, while false positives can unnecessarily block irrigation.

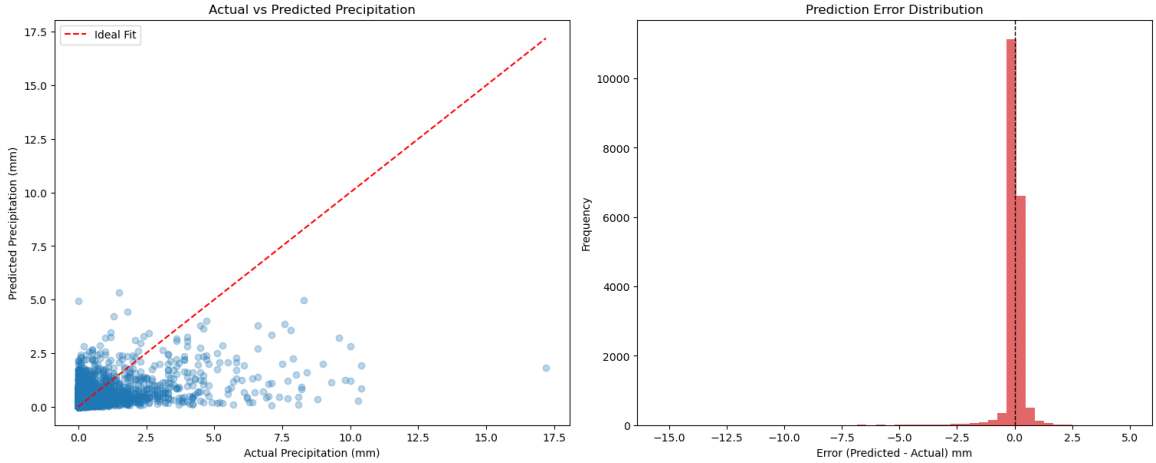

Figure 5.4: CNN actual-versus-predicted rainfall and prediction error distribution. This figure is useful for comparing the sequence model with the deployed MLP, showing whether the 24-hour input history improves peak rainfall estimation or reduces error spread.

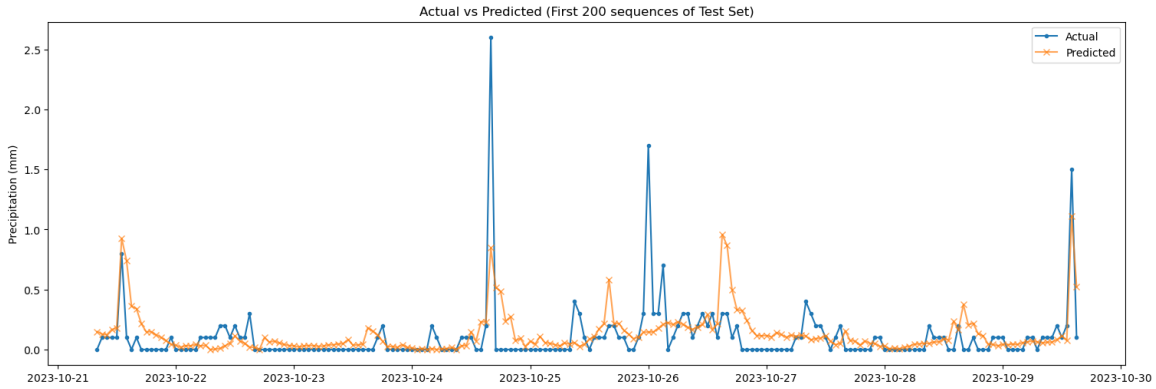

Figure 5.5: CNN actual and predicted rainfall over the first 200 test sequences. The graph helps evaluate whether temporal context allows the CNN to follow changes in rainfall better than the current-weather MLP, which is important if a future firmware version buffers the previous 24 hours of sensor data.

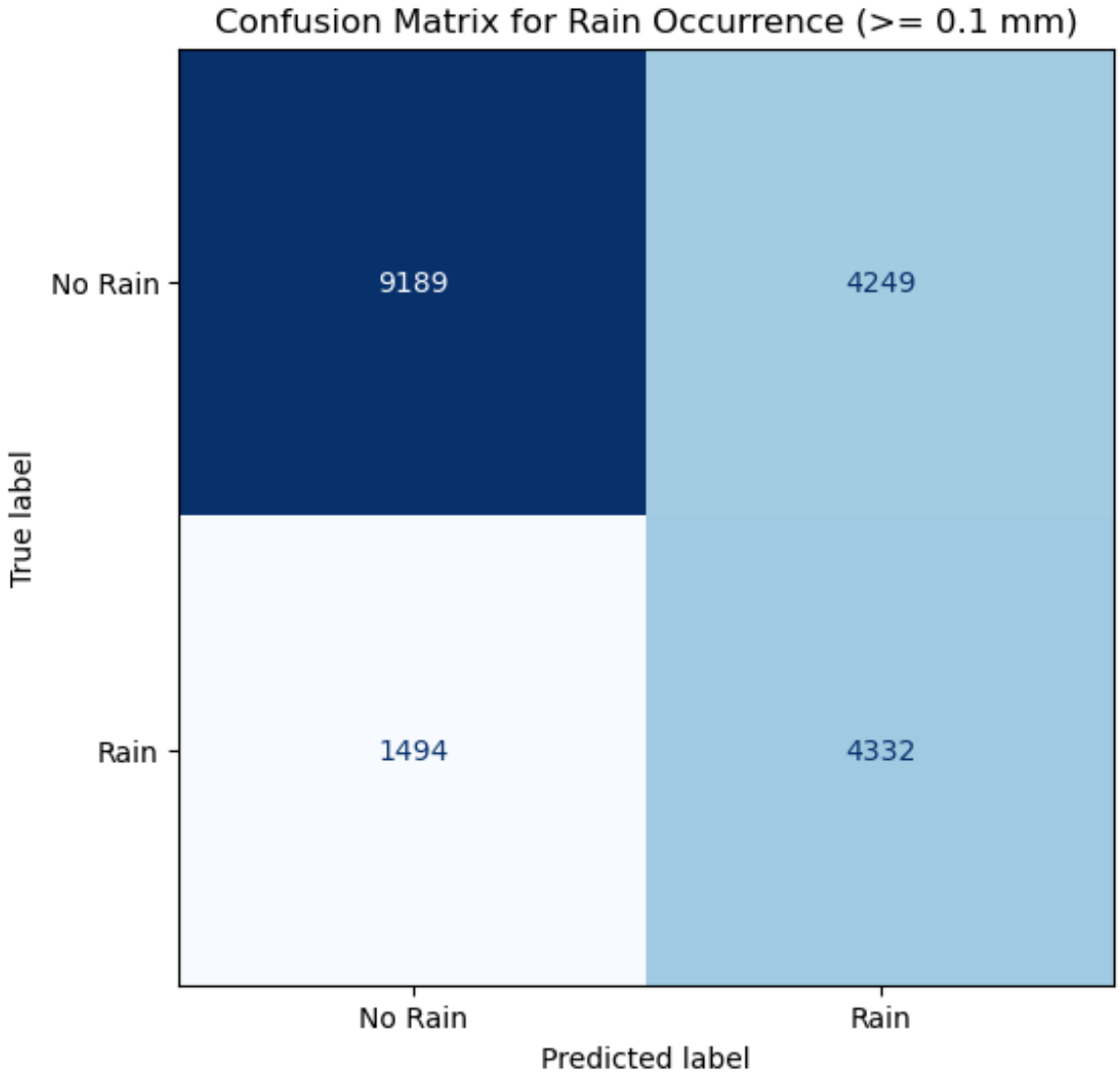

Figure 5.6: CNN rain-occurrence confusion matrix. This graph supports evaluation of the CNN as a future deployment option by showing its rain/no-rain decision behaviour, not only its rainfall amount RMSE and R^2.

5.3 Embedded Firmware Results

The model deployment pipeline produced a 5,172-byte TensorFlow Lite rainfall model for the MLP. The master firmware uses a 48 KB tensor arena and registers only the fully connected operator required by the deployed network. This is an efficient embedded design because unused TensorFlow Lite Micro operators are not included in the resolver.

The firmware implements the following functions successfully at source-code level:

(i) Sensor node construction and transmission of weather packets every 5 s.

(ii) Sensor node calculation of vapour pressure deficit from temperature and relative humidity.

(iii) Sensor node heartbeat transmission every 15 s.

(iv) Section node reception of sensor packets and forwarding to the master.

(v) Section node GPIO actuation for valve, spray and fertilizer outputs.

(vi) Master node reception and caching of weather samples.

(vii) Master node initialization of the TensorFlow Lite Micro interpreter.

(viii) Master node rainfall inference and command transmission every 30 s.

Serial-monitor screenshots are useful in this project because they show the system operating in real time. The recommended screenshots are the sensor-node output while transmitting weather values, the section-node output while forwarding received packets and driving actuators, and the master-node output while initializing the rainfall model and printing the predicted rainfall amount.

Image placeholder

Add image file: `images/sensor_serial_output.png`

Recommended evidence: a screenshot of the Arduino IDE Serial Monitor or another serial terminal showing the sensor node sending weather packets, including temperature, humidity, vapour pressure deficit, soil temperature, soil moisture, ET_0 and shortwave radiation.

Figure 5.7: Sensor-node serial-terminal output during weather packet transmission. The output verifies that the node packages weather variables and sends them through ESP-NOW at the configured interval.

Image placeholder

Add image file: `images/section_serial_output.png`

Recommended evidence: a screenshot showing the section node receiving a weather packet from a sensor node, forwarding it to the master and reporting valve, spray or fertilizer state after receiving a control command.

Figure 5.8: Section-node serial-terminal output during packet forwarding and actuator control. The log confirms that the intermediate farm section node receives local data, forwards it to the master and responds to control commands.

Image placeholder

Add image file: `images/master_serial_output.png`

Recommended evidence: a screenshot showing the master node printing "Rainfall ML model ready", receiving weather values, printing the ML precipitation forecast and sending a control packet to the section node.

Figure 5.9: Master-node serial-terminal output during rainfall inference and irrigation command generation. The output demonstrates that the TensorFlow Lite Micro model is initialized, the rainfall amount is predicted locally and a section-specific control packet is transmitted.

The system is modular because additional sensor nodes can be added by assigning node IDs and peer MAC addresses. The section node can also be replicated for separate farm zones. This design is important for irrigation because different plots may require different control commands.

5.4 Discussion

The project demonstrates that edge AI can be integrated with low-cost microcontroller irrigation control. The main strength of the system is that rainfall inference is performed locally

on the master ESP32, reducing dependence on cloud services during operation. The ESP-NOW architecture also avoids the need for a Wi-Fi router in the field, which is useful for short-range farm section networks.

The use of ET_0, shortwave radiation and vapour pressure deficit connects the control decision to physical water loss processes. ET_0 rises when atmospheric demand is high, radiation indicates available energy for evaporation and vapour pressure deficit summarizes atmospheric drying demand. The humidity threshold reduces irrigation during moist atmospheric conditions. The rainfall model adds a predictive component so that irrigation can be blocked when rain is expected.

There are also limitations. The model was trained on historical weather data from a gridded API source rather than on field measurements from the exact deployed sensors. The sensor firmware currently uses a recent weather sample header for prototype transmission rather than a complete calibrated sensor suite. The deployed MLP predicts rainfall amount from current weather only, so it cannot fully capture larger-scale storm dynamics. The irrigation duration is based on ET_0 thresholds rather than a calibrated crop-water balance with crop type and flow rate. These limitations do not invalidate the prototype, but they identify the next steps required for field-ready deployment.

5.5 Conclusion

This project designed and implemented a prototype ESP32-based smart irrigation system that combines weather data, embedded machine-learning rainfall forecasting and local actuator control. Historical hourly weather data from 2015 to 2025 were processed and used to train two neural rainfall models. A compact MLP model was selected and deployed through TensorFlow Lite Micro on the master ESP32. The firmware architecture supports sensor, section and master nodes communicating through ESP-NOW, with section-level actuator outputs for irrigation, pesticide and fertilizer control.

The deployed rainfall model achieved an hourly rainfall MAE of 0.136 mm and RMSE of 0.484 mm on the test data. The control algorithm uses the predicted rainfall amount to block irrigation when rainfall is expected, and uses ET_0, shortwave radiation and humidity to enable irrigation under high evaporative demand. The study therefore achieved its objective of developing a low-

cost, edge-AI irrigation control prototype.

5.6 Recommendations

Future work should replace the prototype Open-Meteo soil moisture samples with physical soil moisture sensors calibrated for the target soil type. This would allow the system to combine atmospheric demand, rainfall prediction and actual root-zone water status from local measurements. The irrigation duration should also be calibrated using valve flow rate, plot area, crop coefficient and soil water holding capacity.

The CNN sequence model should be considered for future firmware work because it achieved better RMSE and R^2 than the deployed MLP. To deploy it efficiently, the master node would need reliable buffering of the previous 24 hours of weather features and careful memory testing. Model quantization could further reduce memory use and improve inference speed.

The ESP-NOW network should be tested in an actual farm environment to measure packet loss, range, power consumption and interference. Security options should also be reviewed before field deployment, especially if fertilizer or pesticide actuation is enabled. Finally, a long-term field trial should compare water use, crop response and reliability against manual or timer-based irrigation.

REFERENCES

Allen, R. G., Pereira, L. S., Raes, D., and Smith, M. (1998). *Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements*. FAO Irrigation and Drainage Paper 56. Food and Agriculture Organization of the United Nations, Rome.

David, R., Duke, J., Jain, A., Reddi, V. J., Jeffries, N., Li, J., Kreeger, N., Nappier, I., Natraj, M., Wang, T., Warden, P., and Rhodes, R. (2021). TensorFlow Lite Micro: Embedded machine learning for tinyml systems. *Proceedings of Machine Learning and Systems*, 3:800–811.

Espressif Systems (2026). ESP-NOW programming guide. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html. Accessed 27 May 2026.

Garcia, L., Parra, L., Jimenez, J. M., Lloret, J., and Lorenz, P. (2020). IoT-based smart irrigation systems: An overview on the recent trends on sensors and IoT systems for irrigation in precision agriculture. *Sensors*, 20(4):1042.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.

Hersbach, H. et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730):1999–2049.

Kamienski, C., Soininen, J.-P., Taumberger, M., Dantas, R., Toscano, A., Salmon Cinotti, T., Filev Maia, R., and Torre Neto, A. (2019). Smart water management platform: IoT-based precision irrigation for agriculture. *Sensors*, 19(2):276.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.

Liakos, K. G., Busato, P., Moshou, D., Pearson, S., and Bochtis, D. (2018). Machine learning in agriculture: A review. *Sensors*, 18(8):2674.

Open-Meteo (2026). Historical weather API. https://open-meteo.com/en/docs/historical-weather-api. Accessed 27 May 2026.